

Pipelines CI / CD — architecture à deux canaux

Hublot utilise une **CI centralisée** (GitHub Actions) et des **CD distincts** selon la cible. Ce n’est pas un pipeline monolithique unique : c’est le modèle courant **GitHub + Netlify**.

Vue d’ensemble



Workflow	Fichier	Rôle
CI	<code>.github/workflows/ci.yml</code>	Qualité : lint, tests, audit, build, E2E
CD Netlify	<code>.github/workflows/cd-netlify.yml</code>	Optionnel : POST build hook après CI

Workflow	Fichier	Rôle
CD NAS	<code>.github/workflows/cd-nas.yml</code>	Optionnel : rsync SSH si secrets configurés
Audit planifié	<code>.github/workflows/audit-scheduled.yml</code>	Chaque lundi — rapport npm

CI (obligatoire)

Déclenchée sur `main`, `staging` et les pull requests.

Job	Bloquant
ESLint	Oui
Tests unitaires (48)	Oui
Audit production (<code>npm run audit:prod</code>)	Oui (critical + high hors allowlist)
Build <code>build/</code>	Oui
E2E Playwright (3 scénarios)	Oui

CD Netlify (cloud)

Mécanisme principal : webhook Git → Netlify build (`netlify.toml`).

Gate recommandée (sans double build) :

1. Netlify → **Site configuration** → **Build & deploy** → **Deploy contexts**
2. **Production branch** → **Deploy only when required checks pass**
3. Cocher le check `CI` (workflow GitHub Actions)

Mécanisme optionnel : secret `NETLIFY_BUILD_HOOK` + workflow `cd-netlify.yml` pour déclencher un build **après** CI verte. Si vous utilisez le hook, désactivez le build automatique sur chaque push dans Netlify pour éviter deux builds.

CD NAS (réseau interne)

Mode	Comment
Manuel	<code>./scripts/deploy-nas.sh</code> puis copie / Docker sur le NAS
Semi-auto	GitHub → Actions → CD NAS Synology (secrets <code>NAS_HOST</code> , <code>NAS_USER</code> , <code>NAS_PATH</code> , <code>NAS_SSH_KEY</code>)
Rsync local	<code>NAS_HOST=... NAS_USER=... NAS_PATH=... ./scripts/deploy-nas.sh --rsync</code>

Le compose NAS monte `./build` (voir `docker-compose.yml`).

Environnement staging

Voir [ENVIRONNEMENT_STAGING.md](./1.2 Préparer le déploiement d'une application/1.2.3 Mettre en place un environnement de test.md) — branche `staging` + deploy previews Netlify.

Fichiers liés

- [DEVOPS.md](./1.1 Les bases de la démarche DevOps/1.1.2 La démarche DevOps.md)
- [ARCHITECTURE_DEPLOIEMENT.md](./1.1 Les bases de la démarche DevOps/1.1.4 La mise en place de l'intégration continue (CI).md)
- [MONITORING.md](./1.1 Les bases de la démarche DevOps/1.1.2.1 Monitoring et supervision.md)
- [SECURITE_AUDIT.md](./1.2 Préparer le déploiement d'une application/1.2.4.3 Audit des dépendances.md)